

MÓDULO 1 · PIPELINE DE DATOS Y PREPARACIÓN PROFESIONAL

Exploración Avanzada de Datos (EDA)

Material teórico — Pandas, Seaborn y YData Profiling

Objetivo del material

Comprender cómo realizar una Exploratory Data Analysis (EDA) para conocer la estructura, calidad, distribución y relaciones presentes en un conjunto de datos antes de construir modelos de Machine Learning.

1. ¿Qué es la Exploración de Datos?

La **Exploratory Data Analysis (EDA)**, o Exploración Exploratoria de Datos, es el proceso mediante el cual se investiga un conjunto de datos antes de utilizarlo para construir un modelo de Machine Learning.

La EDA permite conocer qué información contiene el dataset, cómo están representadas las variables, qué problemas de calidad existen y qué patrones o relaciones pueden observarse.

No se trata simplemente de generar gráficos. Una EDA debe ayudar a responder preguntas y tomar decisiones sobre las etapas posteriores del proyecto.

2. Objetivos de una EDA

- Comprender la estructura del conjunto de datos.
- Identificar variables numéricas y categóricas.
- Detectar valores faltantes.
- Detectar registros duplicados.
- Encontrar valores inesperados o inconsistentes.
- Analizar la distribución de las variables.
- Identificar posibles valores atípicos.
- Explorar relaciones entre variables.
- Detectar posibles problemas antes del modelado.
- Generar hipótesis que orienten el proyecto de Machine Learning.

3. EDA dentro de un proyecto de Machine Learning

La exploración de datos se encuentra principalmente en la fase de **Comprensión de los datos** de CRISP-DM, aunque sus resultados influyen en las fases posteriores.

Etapas	Qué se analiza	Preguntas orientadoras
1. Conocer el dataset	Filas, columnas, tipos de datos y estructura general.	¿Qué datos tenemos? ¿Cuántos registros existen?
2. Calidad de datos	Valores faltantes, duplicados, inconsistencias y formatos.	¿Los datos están completos? ¿Hay errores evidentes?

Etapa	Qué se analiza	Preguntas orientadoras
3. Estadística descriptiva	Media, mediana, desviación estándar, mínimos y máximos.	¿Cómo se distribuyen los valores?
4. Visualización	Gráficos de distribución, relaciones y categorías.	¿Qué patrones podemos observar visualmente?
5. Relaciones	Correlaciones y asociaciones entre variables.	¿Qué variables parecen estar relacionadas?

4. Primer contacto con un dataset

Cuando se recibe un dataset, el primer paso no debería ser entrenar un modelo. Primero debemos conocer qué tenemos.

Con **Pandas** podemos cargar y explorar rápidamente los datos.

```
import pandas as pd

df = pd.read_csv('clientes.csv')
print(df.head())
```

La función **head()** permite observar las primeras filas del dataset y obtener una primera impresión de su contenido.

5. Conocer la estructura del dataset

shape devuelve la cantidad de filas y columnas.

```
print(df.shape)
```

Por ejemplo, si el resultado es **(1000, 8)**, significa que tenemos 1000 registros y 8 columnas.

info() permite conocer los nombres de las columnas, cantidad de valores no nulos y tipos de datos.

```
df.info()
```

columns permite consultar los nombres de las columnas.

```
print(df.columns)
```

6. Tipos de variables

Identificar correctamente el tipo de cada variable es fundamental para decidir qué análisis y transformaciones serán necesarios.

- **Numéricas:** edades, precios, cantidades, ingresos.
- **Categorías:** ciudad, tipo de cliente, categoría.
- **Binarias:** sí/no, 0/1, abandono/no abandono.
- **Fechas:** fecha de compra, fecha de registro.
- **Texto:** comentarios, descripciones o mensajes.

Pandas puede mostrar los tipos mediante:

```
print(df.dtypes)
```

7. Estadística descriptiva

La estadística descriptiva permite resumir las principales características de las variables numéricas.

```
df.describe()
```

Entre los valores que podemos encontrar se encuentran:

- **count:** cantidad de valores disponibles.
- **mean:** promedio.
- **std:** desviación estándar.
- **min:** valor mínimo.
- **25%:** primer cuartil.
- **50%:** mediana.
- **75%:** tercer cuartil.
- **max:** valor máximo.

La estadística descriptiva no reemplaza la visualización. Ambas herramientas se complementan.

8. Valores faltantes

Un valor faltante aparece cuando una observación no posee información para una determinada variable.

Antes de decidir cómo tratarlos, debemos conocer su cantidad y distribución.

```
df.isnull().sum()
```

También podemos calcular el porcentaje de valores faltantes:

```
porcentaje = df.isnull().mean() * 100  
print(porcentaje)
```

La EDA permite detectar el problema. La decisión sobre cómo imputar o eliminar esos valores corresponde a la etapa de preparación de datos.

9. Registros duplicados

Los registros duplicados pueden distorsionar análisis y modelos si representan accidentalmente la misma observación.

```
df.duplicated().sum()
```

Encontrar duplicados no significa eliminarlos automáticamente. Primero debe determinarse si son realmente errores o si representan observaciones válidas.

10. Distribuciones

Una distribución muestra cómo se concentran los valores de una variable.

Por ejemplo, al analizar la edad de clientes podemos encontrar una concentración entre determinados valores y pocos registros en los extremos.

Las distribuciones ayudan a detectar:

- asimetrías;
- concentraciones de valores;
- rangos inesperados;
- valores extremos;

- posibles errores de carga.

11. Valores atípicos (Outliers)

Un **outlier** es una observación que se encuentra muy alejada del comportamiento predominante de una variable.

Un valor atípico no necesariamente es un error. Puede representar una situación real e importante.

Por ejemplo, si analizamos ingresos mensuales, una persona con ingresos muy superiores al resto puede ser un caso real.

Por este motivo, los outliers deben investigarse antes de decidir eliminarlos.

12. Visualización con Seaborn

Seaborn es una biblioteca de Python construida sobre Matplotlib que facilita la creación de visualizaciones estadísticas.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

La visualización permite detectar patrones que pueden ser difíciles de identificar observando únicamente una tabla.

Gráfico	Uso principal	Ejemplo
Histograma	Analizar la distribución de una variable numérica.	Edad, ingresos, precio.
Boxplot	Observar dispersión y posibles valores atípicos.	Ingresos por categoría.
Countplot	Contar observaciones por categoría.	Clientes por tipo de plan.
Scatterplot	Observar la relación entre dos variables numéricas.	Edad e ingresos.
Heatmap	Representar matrices de correlación.	Correlación entre variables numéricas.

13. Análisis univariado

El análisis univariado estudia una variable a la vez.

Por ejemplo, podemos analizar la distribución de la edad de los clientes.

```
sns.histplot(data=df, x='edad', kde=True)
plt.show()
```

14. Análisis bivariado

El análisis bivariado estudia la relación entre dos variables.

Por ejemplo, podemos analizar la relación entre edad e ingresos:

```
sns.scatterplot(data=df, x='edad', y='ingresos')
plt.show()
```

El objetivo no es afirmar automáticamente que una variable cause a la otra. Una relación observada no implica causalidad.

15. Correlación

La correlación permite medir la intensidad y dirección de una relación lineal entre variables numéricas.

```
correlaciones = df.corr(numeric_only=True)
print(correlaciones)
```

Una correlación cercana a 1 indica una relación lineal positiva fuerte. Una cercana a -1 indica una relación lineal negativa fuerte. Una cercana a 0 indica poca relación lineal.

La correlación no demuestra causalidad.

16. Matriz de correlación y Heatmap

Una matriz de correlación puede visualizarse mediante un mapa de calor o **heatmap**.

```
corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```

Este gráfico permite identificar rápidamente pares de variables que presentan relaciones lineales fuertes.

17. YData Profiling

YData Profiling permite generar automáticamente un informe exploratorio de un dataset.

La herramienta resume información estadística, tipos de variables, valores faltantes, distribuciones, correlaciones y posibles advertencias.

Instalación:

```
pip install ydata-profiling
```

Uso básico:

```
from ydata_profiling import ProfileReport

profile = ProfileReport(df, title='Reporte EDA')
profile.to_file('reporte_eda.html')
```

El informe generado puede abrirse en un navegador y utilizarse como apoyo para analizar el dataset.

18. Ventajas y límites de la automatización

Las herramientas automáticas son útiles para acelerar el diagnóstico inicial, pero no sustituyen el análisis humano.

- **Ventaja:** permiten revisar rápidamente muchas variables.
- **Ventaja:** detectan posibles problemas de calidad.
- **Ventaja:** facilitan la generación de estadísticas y gráficos.
- **Límite:** no conocen el contexto del problema de negocio.
- **Límite:** una advertencia automática debe ser interpretada.
- **Límite:** no todas las correlaciones tienen valor práctico.

19. EDA y toma de decisiones

El resultado más importante de una EDA no es el gráfico en sí mismo, sino la decisión que puede fundamentarse a partir de él.

Ejemplo

Si observamos que una variable tiene un 45 % de valores faltantes, no basta con registrar el porcentaje. Debemos preguntarnos por qué ocurre, si la variable es importante y qué estrategia será adecuada para tratarla.

20. Buenas prácticas para una EDA profesional

- No comenzar el modelado sin conocer los datos.
- Documentar las decisiones tomadas durante la exploración.
- Combinar análisis estadístico y visualización.
- No eliminar datos automáticamente sin investigar su significado.
- Distinguir entre problemas reales y comportamientos válidos.
- Analizar las variables en relación con el problema de negocio.
- Evitar conclusiones causales a partir de simples correlaciones.
- Repetir la exploración cuando se incorporen nuevos datos.

21. Ejemplo de flujo EDA con Pandas y Seaborn

Un flujo inicial podría ser:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('clientes.csv')

print(df.head())
print(df.shape)
df.info()
print(df.describe())
print(df.isnull().sum())
print(df.duplicated().sum())

sns.histplot(data=df, x='edad', kde=True)
plt.show()
```

Este código no constituye una EDA completa, pero muestra una secuencia inicial de inspección.

22. Relación entre EDA y preparación de datos

La EDA permite descubrir problemas que posteriormente deberán resolverse durante la preparación de los datos.

Hallazgo en EDA	Posible acción posterior
Valores faltantes	Imputación o eliminación justificada.
Variables categóricas	Codificación adecuada.
Escalas muy diferentes	Escalado cuando el algoritmo lo requiera.
Outliers	Investigar, transformar o tratar según contexto.

Hallazgo en EDA	Posible acción posterior
Clases desbalanceadas	Aplicar estrategias específicas de balanceo.

23. Idea central

Una buena EDA permite conocer los datos antes de pedirles que aprendan algo.

El objetivo es transformar un conjunto de datos desconocido en información comprensible que permita tomar decisiones fundamentadas durante el proyecto de Machine Learning.

24. Glosario

Concepto	Definición
EDA	Exploratory Data Analysis o análisis exploratorio de datos.
Dataset	Conjunto estructurado de datos.
Feature	Variable utilizada como entrada para un modelo.
Target	Variable que el modelo intenta predecir.
Outlier	Observación alejada del comportamiento habitual.
Correlación	Medida de asociación lineal entre variables.
Distribución	Forma en que se concentran los valores.
Pandas	Biblioteca de Python para manipulación y análisis de datos.
Seaborn	Biblioteca de Python para visualización estadística.
YData Profiling	Herramienta para generar informes automáticos de exploración.